

¿Cuáles son los tipos de diagrama más relevantes en UML? (Describe al menos 4)

1. Diagrama de Clases

- Representa la estructura estática del sistema mostrando clases, atributos, métodos y relaciones (herencia, asociación, composición, etc.).
- Es esencial en el modelado orientado a objetos.

2. Diagrama de Casos de Uso

- Muestra las interacciones entre los actores (usuarios u otros sistemas) y el sistema.
- Se usa para especificar los requisitos funcionales.

3. Diagrama de Secuencia

- Representa cómo los objetos interactúan entre sí a través del tiempo, mediante mensajes.
- Útil para detallar la lógica de casos de uso.

4. Diagrama de Actividades

- Representa el flujo de trabajo o procesos, parecido a un diagrama de flujo.
- Ideal para describir procesos de negocio o lógica de un algoritmo.

¿Cómo representar instancias de las clases en UML?

Las **instancias de clases** se representan con **objetos**. En UML, un objeto se muestra como un rectángulo con dos partes:

- El nombre del objeto (subrayado) y su tipo (**nombreObjeto:NombreClase**)
- Sus valores de atributos (opcional)

Ejemplo:

Clase:

uml

```
Clase: Persona
-----
+ nombre: String
+ edad: Integer
-----
+ saludar(): String
```

Instancia del objeto:

uml

```
juan:Persona
-----
nombre = "Juan"
edad = 30
```

Se dibuja así en UML:

markdown

```
-----
juan : Persona
-----
nombre = "Juan"
edad = 30
-----
```

¿UML permite incluir notas, cómo y para qué?

Sí, UML **permite incluir notas** para agregar aclaraciones, documentación o comentarios que **no forman parte del modelo lógico** pero son útiles para entenderlo.

- Se representan como un **rectángulo con una esquina doblada** (como una nota adhesiva).
- Se pueden conectar a cualquier elemento UML mediante una **línea discontinua**.

¿Para qué?

- Documentar decisiones de diseño
- Aclarar el propósito de una clase, atributo o relación
- Agregar referencias externas o reglas de negocio

¿Qué es un método constructor y para qué se utiliza?

Un **método constructor** es una función especial de una clase que **se ejecuta automáticamente cuando se crea un objeto** de esa clase.

Funciones del constructor:

- Inicializar atributos del objeto
- Configurar el estado inicial

¿Qué tipos de relaciones pueden especificarse entre clases?

UML permite representar varios tipos de relaciones entre clases:

1. Asociación

- Representa una relación entre dos clases.
- Puede tener multiplicidad (1..1, 1..*, etc.).

2. Herencia (Generalización)

- Una clase hija hereda atributos y métodos de una clase padre.

3. Agregación

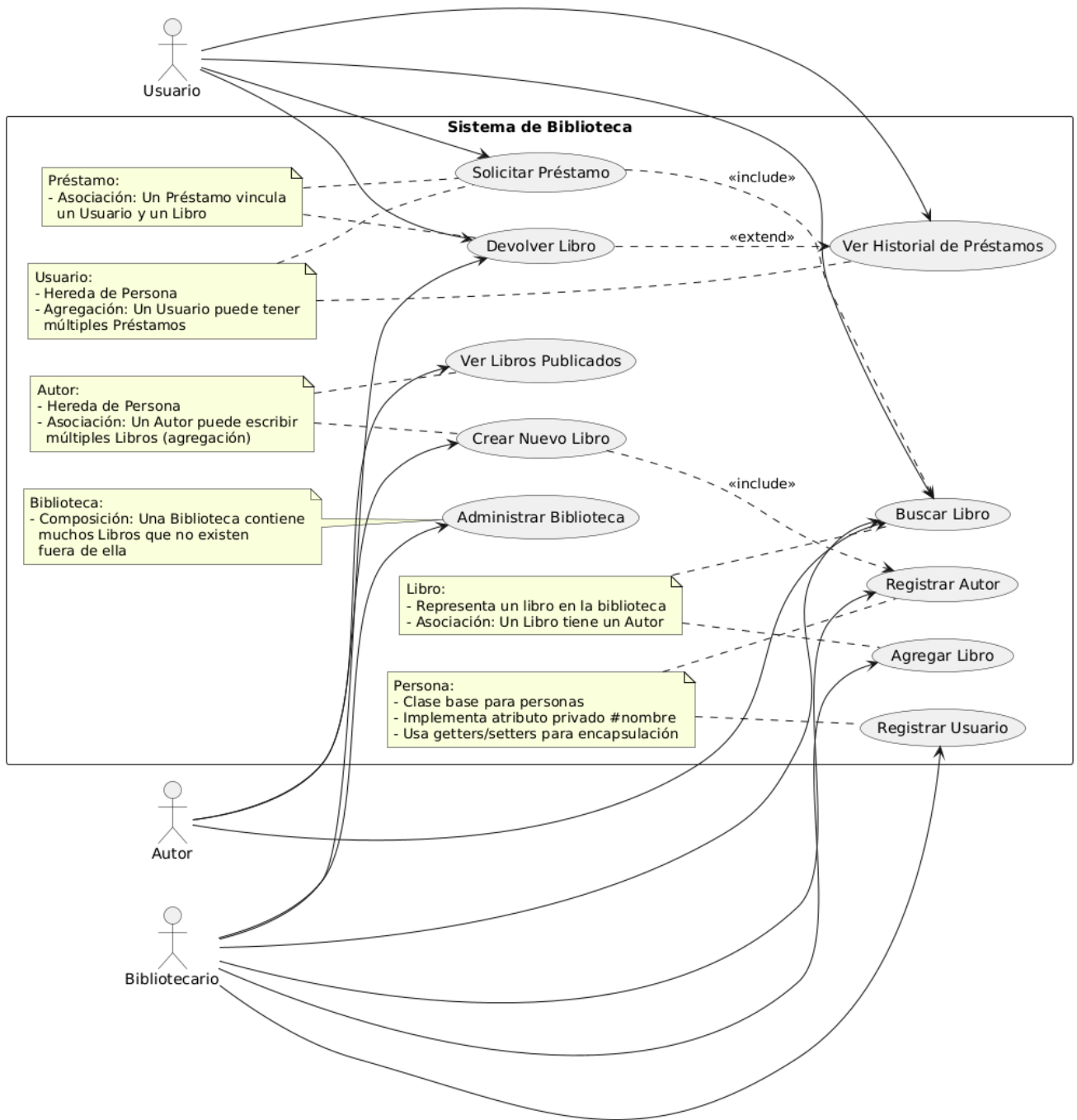
- Una relación de "tiene un" (whole-part), donde las partes pueden existir independientemente.

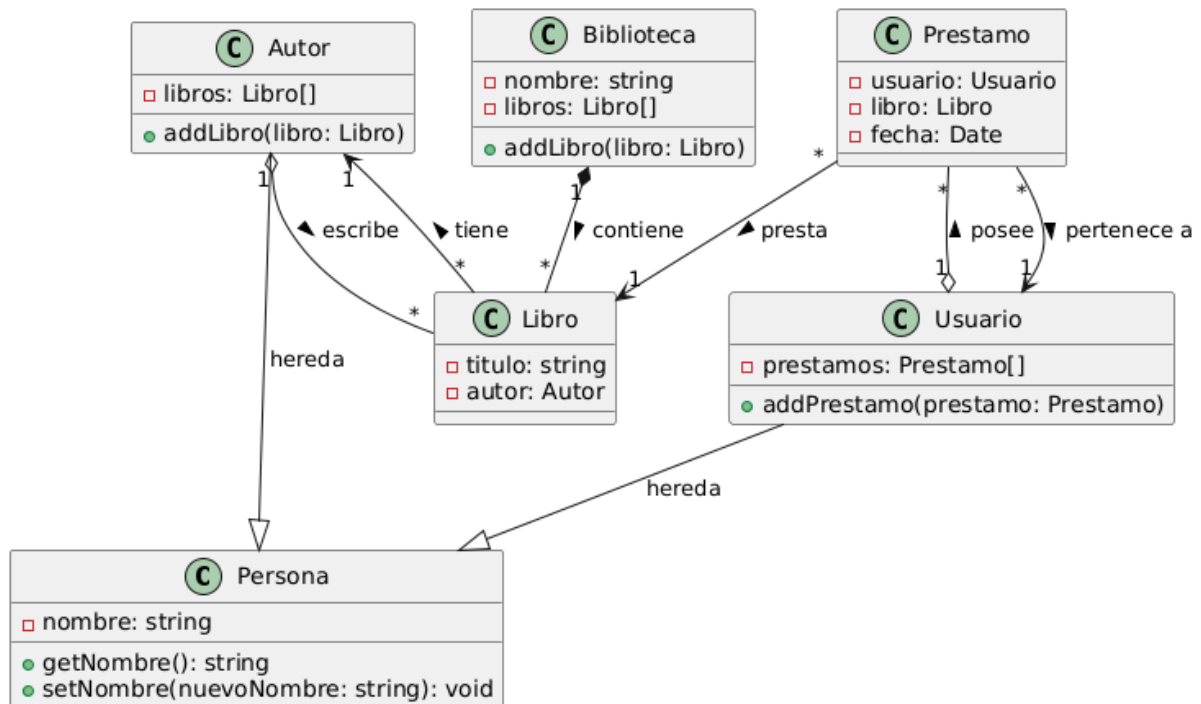
4. Composición

- Similar a la agregación, pero con una relación más fuerte; si el objeto compuesto muere, sus partes también.

5. Dependencia

- Una clase usa otra temporalmente (por ejemplo, como parámetro de un método).





glosario:

Lenguaje Unificado de Modelado. Es un lenguaje gráfico estándar para visualizar, especificar, construir y documentar los artefactos de un sistema de software.

POO:

Programación Orientada a Objetos. Paradigma de programación basado en el uso de objetos, que combina datos y comportamientos.

Objeto:

Entidad que contiene atributos (datos) y métodos (comportamientos). Es una instancia de una clase.

Clase:

Molde o plantilla para crear objetos. Define los atributos y métodos comunes a todos sus objetos.

Modelo:

Representación abstracta de un sistema, utilizada para entender, diseñar o simular su comportamiento.

Atributo:

Propiedad o característica de una clase u objeto. Representa información que describe al objeto.

Alcance:

Nivel de visibilidad o accesibilidad de una variable, método o clase (por ejemplo: público, privado, protegido).

Parámetro:

Variable que se utiliza para recibir información en un método o función.

Agregación:

Tipo de relación entre clases donde una representa una parte de otra, pero puede existir independientemente (relación "tiene un").

Herencia:

Mecanismo por el cual una clase puede heredar atributos y métodos de otra clase.

Polimorfismo:

Capacidad de los objetos para responder de forma diferente al mismo mensaje (método), dependiendo del tipo de objeto.

Extensión:

Ampliación de la funcionalidad de una clase o componente, generalmente a través de la herencia o agregación.

Función:

Bloque de código reutilizable que realiza una tarea específica y puede retornar un valor.

Relación:

Conexión entre dos o más clases que describe cómo interactúan entre sí (por ejemplo: asociación, herencia, dependencia).

Abstracción:

Principio que consiste en enfocarse en los aspectos esenciales de un objeto, ocultando los detalles innecesarios.

Sobrecarga:

Definición de múltiples métodos con el mismo nombre pero diferentes parámetros en una misma clase.